



Industrial Bid Management System

Full-Stack Platform with AI · Real-time Maps · WebSocket · JWT Security



PROJECT	DATE	TYPE
BidFlow v1.0.0	June 2026	Full-Stack Web App



Project Documentation

This document provides a comprehensive technical overview of the BidFlow AuctionX Tracking platform — covering architecture, technology stack, features, APIs, database design, security, and deployment.



Table of Contents

1	Project Overview	2	System Architecture
3	Technology Stack	4	Backend — Spring Boot
5	Frontend — React.js	6	Database — MongoDB
7	API Documentation	8	Authentication & Security
9	AI Features	10	User Roles & Permissions
11	Bid Workflow & Lifecycle	12	Real-time Features
13	Live Map Tracking	14	Payment & Subscriptions
15	Deployment (Render + Docker)	16	Multilingual & Theme

1. Project Overview

BidFlow AuctionX Tracking is a production-grade, full-stack industrial bid management platform. It enables organizations to manage the complete lifecycle of business bids — from client submission through AI-powered analysis, employee assignment, progress tracking, to final approval and client feedback.



AI-Powered Analysis

Google Gemini AI for win probability prediction, sentiment analysis, bid summaries, and intelligent recommendations.



Live Map Tracking

Real-time location tracking of team members using OpenStreetMap and React Leaflet with geolocation API.



Real-time Notifications

WebSocket-based instant notifications for every bid action — assignments, approvals, status changes.



Enterprise Security

JWT authentication, BCrypt password hashing, Google OAuth 2.0, and role-based access control.



Subscription System

4-tier plan system (Free/Basic/Pro/Enterprise) with payment processing and invoice generation.



Multilingual Support

Full English, Hindi, and Arabic language support with RTL layout for Arabic and persistent preference.

2. System Architecture

BidFlow follows a **3-tier client-server architecture** with clear separation between the presentation layer (React frontend), business logic layer (Spring Boot backend), and data layer (MongoDB Atlas). All communication is via RESTful HTTP APIs with JWT authentication.

PRESENTATION LAYER – CLIENT BROWSER

React 19 + Vite

Zustand State

React Router DOM

Axios HTTP Client

Tailwind CSS v4

Recharts Analytics

React Leaflet Map

↕ HTTP/REST (Axios) • JWT Bearer Token • WebSocket (STOMP)

BUSINESS LOGIC LAYER – SPRING BOOT BACKEND (PORT 8080)

REST Controllers

Spring Security + JWT

Service Layer

GeminiService (AI)

NotificationService

WebSocket Handler

OtpService (Email)

SSHService

↕ MongoDB Java Driver (TLS/SSL)

DATA LAYER – MONGODB ATLAS (CLOUD, AWS MUMBAI)

users collection

bids collection

notifications

payments

subscription_plans

discussions

meetings

otp_tokens

↕ HTTPS API Calls

 EXTERNAL SERVICES

Google Gemini AI API

Google OAuth 2.0

OpenStreetMap / Nominatim

Gmail SMTP (OTP Email)

Render.com (Hosting)

11

REST Controllers

8

MongoDB Collections

40+

API Endpoints

4

User Roles

3

Languages Supported

4

Subscription Plans

Communication Protocols

REST/HTTP: All CRUD operations use standard HTTP methods (GET, POST, PUT, PATCH, DELETE) with JSON payloads via Axios on the frontend.

WebSocket (STOMP): Real-time bi-directional communication for live notifications and the SSH terminal feature. Spring Boot's WebSocket support handles connection upgrades.

JWT (JSON Web Tokens): Every protected API request carries a signed JWT token in the Authorization header. The backend validates the token on each request via a custom filter chain.

3. Technology Stack — With Definitions

⚙ Backend Technologies

TECHNOLOGY	VERSION	PURPOSE & DEFINITION
Java	17 LTS	Object-oriented programming language. Java 17 is a Long-Term Support release offering records, sealed classes, and improved performance used for the entire backend.
Spring Boot	3.2.5	An opinionated Java framework that auto-configures components. Eliminates boilerplate setup for web servers, databases, and security — letting developers focus on business logic.
Spring Security	6.2.4	Comprehensive authentication and authorization framework. Provides filter chains, CORS configuration, CSRF protection, and role-based access control for all API endpoints.
Spring Data MongoDB	4.2.5	ORM layer for MongoDB. Provides repository interfaces with built-in CRUD, custom query methods, and MongoDB aggregation pipeline support without writing raw queries.
JWT (jjwt)	0.12.5	JSON Web Token library for generating and validating stateless authentication tokens. Tokens are signed with HS256 algorithm and carry user identity information.
OkHttp	4.12.0	Efficient HTTP client for Java. Used by GeminiService to make API calls to Google's Gemini AI endpoint for all AI-powered features in the platform.
Lombok	1.18.x	Annotation processor that auto-generates getters, setters, constructors, and builders at compile time — reducing boilerplate code by up to 70%.

Jackson	2.15.4	Industry-standard JSON serialization/deserialization library. Handles conversion between Java objects and JSON for all API request/response payloads.
Spring WebSocket	6.x	Enables full-duplex communication over a single TCP connection. Used for real-time notifications and the built-in SSH terminal feature.
JSch	0.2.18	Java SSH2 client library (mwiede fork). Powers the SSHService which enables browser-based SSH terminal sessions over WebSocket connections.
Maven	3.x	Java project build tool and dependency manager. Manages all library downloads, compilation, testing, and packaging into a deployable JAR file.



Frontend Technologies

TECHNOLOGY	VERSION	PURPOSE & DEFINITION
React.js	19.x	JavaScript UI library using a virtual DOM and component-based architecture. React 19 introduces enhanced concurrent features and improved server component support.
Vite	8.x	Next-generation frontend build tool using native ES modules. Provides instant Hot Module Replacement (HMR) in development and optimized Rollup bundles for production.
React Router DOM	7.x	Declarative client-side routing library. Enables single-page application navigation with protected routes, nested layouts, and programmatic navigation.
Zustand	5.x	Lightweight global state management library. Manages authentication state, theme, language, bids, and notifications with optional localStorage persistence.
Axios	1.x	Promise-based HTTP client. Configured with interceptors to automatically attach JWT tokens to requests and handle 401 unauthorized responses with auto-logout.

Tailwind CSS	4.x	Utility-first CSS framework. Enables rapid UI development with pre-built classes. Version 4 uses a Vite plugin instead of PostCSS for faster compilation.
React Leaflet	5.x	React wrapper for Leaflet.js map library. Renders interactive OpenStreetMap tiles with custom markers for real-time team location visualization.
Recharts	3.x	Composable charting library built on D3. Used for dashboard analytics: bar charts, line charts, pie charts for bid status distribution and revenue trends.
Framer Motion	12.x	Production-ready animation library for React. Powers page transitions, modal animations, sidebar collapse effects, and micro-interactions throughout the UI.
xterm.js	5.3.0	Full-featured terminal emulator for the browser. Renders the SSH terminal UI and handles keyboard input/output forwarding over WebSocket connections.
React Hot Toast	2.x	Lightweight toast notification library. Displays success, error, and info messages for user actions like bid creation, login, and file uploads.
React Query	5.x	Powerful data fetching and caching library. Handles server state with automatic background refetching, stale-while-revalidate caching, and optimistic updates.

4. Backend — Spring Boot Deep Dive

4.1 Security Layer — JWT Authentication Flow

Every HTTP request passes through a custom **JwtAuthFilter** before reaching any controller. This filter validates the JWT token, loads the user, and sets the Spring Security context.

1 Client sends request with JWT token

Frontend sends HTTP request with `Authorization: Bearer eyJhbGci...` header via Axios interceptor automatically.

2 JwtAuthFilter intercepts

Extracts the token from the Authorization header. Supports both real JWT tokens and demo tokens (demo-token-ROLE format for testing).

3 Token validation

JwtUtil validates the signature, checks expiration, and extracts the userId from the token's subject claim using HS256 verification.

4 User loaded from MongoDB

User document is fetched from MongoDB by ID and wrapped in Spring Security's Authentication object with proper role authorities.

5 SecurityContext populated

SecurityContextHolder stores the authenticated user. Controllers access it via AuthHelper.getCurrentUser() to get the currently logged-in user.

4.2 Controller Architecture



AuthController



BidController

Handles user registration, login, profile management, password change, and Google OAuth integration. Returns JWT tokens on successful authentication.

Full bid CRUD with role-based filtering. Managers see all bids, employees see assigned bids, clients see their own. Includes analytics aggregation pipeline.



AiController

Integrates with Google Gemini AI for chat, sentiment analysis, win probability prediction, bid summaries, smart priority assignment, and duplicate detection.



NotificationController

CRUD for notifications. NotificationService auto-creates notifications on every bid action — assignment, status change, approval, completion.



LocationController

Updates and retrieves user GPS coordinates. Role-based visibility — managers see all locations, employees/clients see only their own.



PaymentController

Manages subscription plans, payment processing, billing cycles (monthly/yearly), invoice generation, and subscription cancellation.

4.3 GeminiService — AI Integration

The **GeminiService** uses OkHttp to call the Groq API (Llama 3.1 model) which is fully compatible with Gemini-style prompting. It builds structured prompts and parses JSON responses for each AI feature.

```
@Service public class GeminiService { // Sends prompt to Groq/Gemini API
via OkHttp public String askGemini(String prompt) { // Build JSON request
body // POST to https://api.groq.com/openai/v1/chat/completions //
Extract text from response choices[0].message.content // Return cleaned
response string } }
```

5. Frontend — React.js Architecture

5.1 Application Routing Structure

The app uses **React Router DOM v7** with nested route layouts. Each role has its own route prefix and layout wrapper with protected route guards that check authentication and redirect if unauthorized.

PATH	ROLE	PAGE DESCRIPTION
/login, /register	—	Public authentication pages
/manager	Manager Admin	Main dashboard with KPI cards, charts, recent bids
/manager/bids	Manager	All bids management with filters, search, bulk actions
/manager/analytics	Manager	Revenue charts, win rate, department breakdown
/manager/ai	Manager	AI Insights — win probability, recommendations
/manager/map	Manager	Live map with all team member locations
/manager/subscription	Manager	Plan management, upgrade/downgrade, billing history
/client	Client	Client dashboard — submitted bids status overview
/client/bids	Client	My bids list with status tracking and feedback
/employee	Employee	Employee dashboard — assigned work, progress

/employee/bids	Employee	Assigned bids with progress update and completion submit
*/map	All roles	Live location map (visibility filtered by role)
*/profile	All roles	User profile, password change, avatar upload
*/notifications	All roles	Notification center with read/delete actions

5.2 State Management — Zustand Stores

Zustand is a minimal, unopinionated state management library. Unlike Redux, it requires no boilerplate providers or reducers. Each store is a simple hook that components subscribe to.

authStore  **Persisted**

- user (object) — full user document
- token (string) — JWT token
- isAuthenticated (boolean)
- login(user, token) action
- logout() action

themeStore  **Persisted**

- theme: 'dark' | 'light'
- toggleTheme() action
- Applies data-theme attribute to DOM
- CSS variables switch on toggle

languageStore  **Persisted**

- language: 'en' | 'hi' | 'ar'
- setLanguage(lang) action
- RTL direction for Arabic
- i18n translations object

notificationStore  **Session**

- notifications[] array
- unreadCount (number)
- fetchNotifications() action
- markAllRead() action

5.3 Key UI Components

DashboardLayout: The main wrapper component for all authenticated pages. Renders a fixed sidebar (left), fixed topbar (top), breadcrumb navigation, and the main content area with scroll.

Sidebar: Role-based navigation links rendered conditionally. Supports collapsible icon-only mode for more screen space. Shows unread notification badge count.

Topbar: Contains a debounced live search bar (350ms), theme toggle button, language switcher dropdown, notification bell with badge, and user avatar with dropdown menu.

AIChatWidget: Floating bottom-right chat button. Opens a slide-up panel with chat history, input field, and real-time responses from the backend AI endpoint.

BidTimeline: Visual timeline component showing each bid tracking stage with completion status, timestamps, and responsible team member names.

6. Database Design — MongoDB Collections

BidFlow uses **MongoDB Atlas** — a cloud-hosted NoSQL document database. MongoDB stores data as flexible JSON-like BSON documents, making it ideal for complex nested objects like bid history, tracking stages, and user profiles without rigid schemas.



users

Core user authentication & profile data

<code>_id</code>	ObjectId	Auto-generated unique identifier by MongoDB
<code>email</code>	String	Unique email address — used as login identifier
<code>password</code>	String	BCrypt hashed password (strength 12) — never stored plain
<code>role</code>	String	CLIENT EMPLOYEE MANAGER ADMIN — controls access
<code>subscriptionPlan</code>	String	FREE BASIC PRO ENTERPRISE — current active plan
<code>latitude / longitude</code>	Double	GPS coordinates updated by live location feature
<code>googleId</code>	String	Google OAuth unique ID — sparse unique index allows null
<code>isActive</code>	Boolean	Soft-delete flag — inactive users cannot login



bids

Core business entity — complete bid lifecycle

bidNumber	String	Auto-generated unique identifier e.g. BID-001-1234
status	String	new → under_review → proposal_generated → awaiting_approval → won/lost
value	Double	Monetary value of the bid in USD
priority	String	high medium low — AI-assigned or manually set
progress	Integer	0–100 percentage of completion updated by employee
aiWinProbability	Integer	0–100 AI-predicted win probability from Gemini
trackingStages	Array	Ordered list of workflow stages with completion timestamps
history	Array	Immutable audit log — every action with user and timestamp
assignedTo	ObjectId	Reference to users collection — assigned employee
clientSentiment	String	positive neutral negative — AI sentiment analysis result

 payments	Subscription payment transaction records	
userId	ObjectId	Reference to user who made the payment
status	String	pending success failed refunded
amount	Double	Payment amount in USD
billingCycle	String	monthly yearly — determines discount applied
invoiceNumber	String	Auto-generated unique invoice number for billing records

paymentMethod

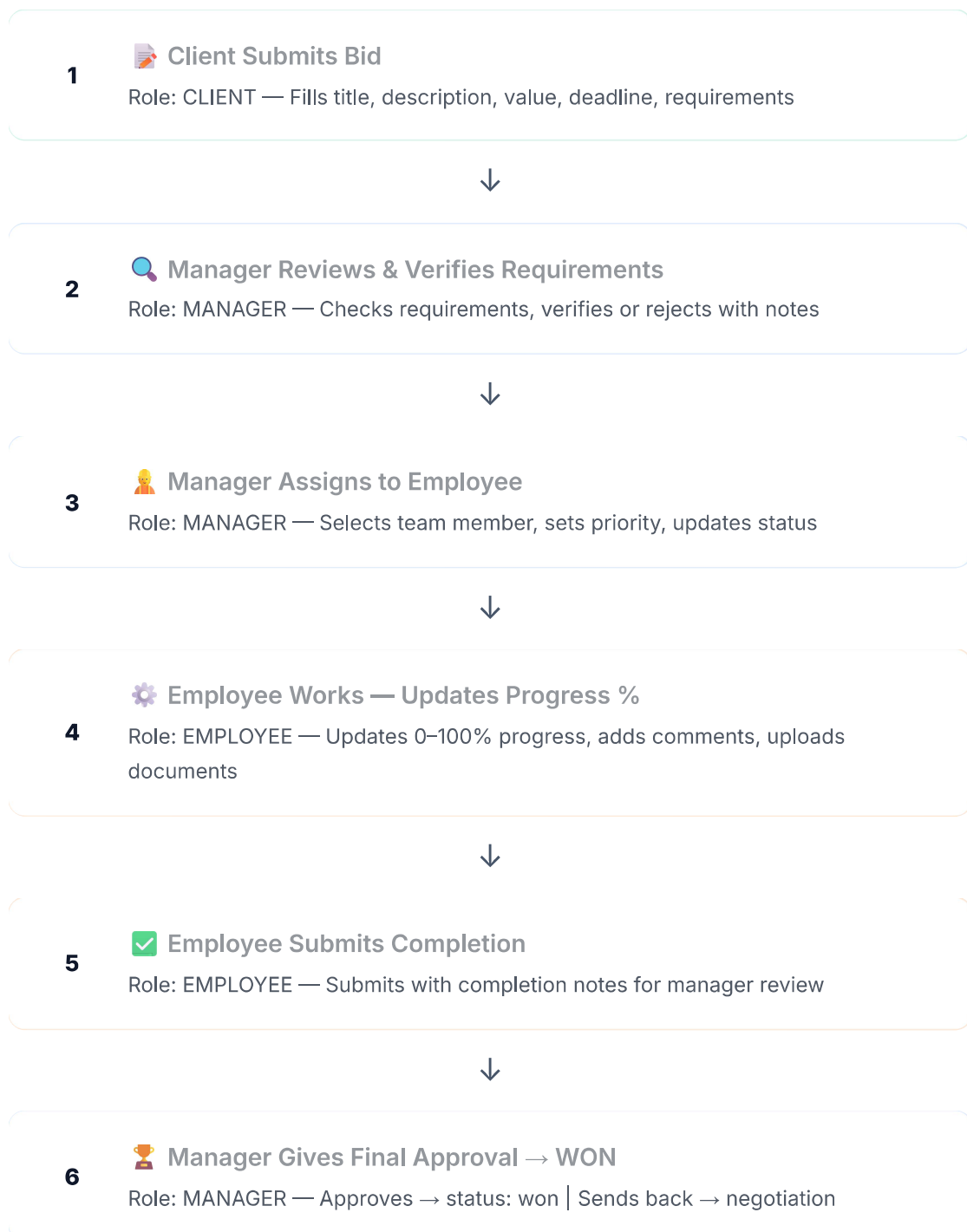
String

card | upi | netbanking — chosen
payment method

7. Bid Management Workflow

The bid lifecycle follows a structured multi-step approval workflow involving three roles. Each stage transition automatically creates notifications and audit history entries.

COMPLETE BID LIFECYCLE



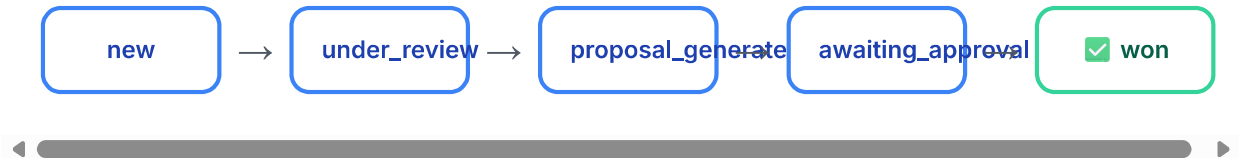


7

★ Client Gives Rating & Feedback

Role: CLIENT — 1–5 star rating with written feedback

Bid Status Flow Diagram



At any stage: can move to **lost** or **negotiation** for revision

8. AI Features — Google Gemini Integration

BidFlow integrates **Groq's Llama 3.1** model (via Gemini-compatible API) to provide intelligent automation across the bid management process.



AI Chat Assistant

Answers questions about bids, stats, win rates, pending items. Has smart local fallback for common queries without API call. Understands English, Hindi greetings.

POST /api/ai/chat



Win Probability Prediction

Analyzes bid value, priority, client sentiment, progress, and deadline to predict 0–100% win probability. Falls back to rule-based scoring if API unavailable.

GET /api/ai/predict/:bidId



Sentiment Analysis

Analyzes client messages and notes to classify sentiment as positive, neutral, or negative with confidence score and keyword extraction.



Executive Summary

Auto-generates a 3–4 sentence professional executive summary for any bid covering scope, value proposition, current status, and key considerations.

POST /api/ai/sentiment

POST /api/ai/summary/:bidId



Smart Priority Assignment

Analyzes bid value, deadline proximity, and client data to automatically assign high/medium/low priority. Saves result back to MongoDB.

POST /api/ai/priority/:bidId



Actionable Recommendations

Returns 4 specific, actionable recommendations to improve win chances for a specific bid based on its data, client history, and current status.

GET /api/ai/recommendations/:bidId

9. Authentication & Security



JWT Token Authentication

JSON Web Tokens signed with HS256 algorithm using a 256-bit secret key. Token carries userId as subject, expires in 7 days (604,800,000ms). Validated on every request by JwtAuthFilter.



BCrypt Password Hashing

All passwords hashed using BCrypt with cost factor 12 — providing ~200ms hash time to resist brute-force attacks. Passwords are never returned in any API response.



Google OAuth 2.0

Full OAuth2 redirect flow. Users click "Sign in with Google" → redirected to Google → callback creates/links account → returns same JWT token as regular login.



Role-Based Access Control

Spring Security method-level security with @PreAuthorize annotations. Each endpoint checks the authenticated user's role before processing the request. 4 roles with granular permissions.



CORS Configuration

Cross-Origin Resource Sharing configured to allow requests only from trusted frontend origins (localhost ports in dev, *.onrender.com in production). Uses pattern matching for wildcard support.



OTP Email Verification

6-digit OTP sent via Gmail SMTP using Spring Mail. OTP tokens stored in MongoDB with TTL expiry. Used for forgot password flow and account verification.

10. User Roles & Permissions Matrix

FEATURE	CLIENT	EMPLOYEE	MANAGER	ADMIN
Submit new bid	✓	✗	✓	✓
View own bids only	✓	✓	✓	✓

View ALL bids	X	X	✓	✓
Assign employee to bid	X	X	✓	✓
Update progress %	X	✓	✓	✓
Submit completion	X	✓	X	X
Final approval (won)	X	X	✓	✓
Give star rating	✓	X	X	X
View analytics dashboard	X	X	✓	✓
Live map — all users	X	X	✓	✓
Delete bid	X	X	✓	✓

11. Deployment Architecture



Developer Machine

Code written in VS Code.
Git Bash used to commit
and push changes to
GitHub repository.

github.com/vishal-q/Bid_AuctionX_Tracking



Backend — Render Web Service

Spring Boot app
containerized with Docker.
Render pulls from GitHub,
builds Docker image, runs
the JAR on port 8080.

bid-auctionx-tracking.onrender.com



Frontend — Render Static Site

Vite builds React app into
static HTML/JS/CSS files.
Render serves them via
CDN. Auto-deploys on
every push to main
branch.

bidflow-frontend-jeeu.onrender.com



Project Summary

BidFlow AuctionX Tracking

A complete, production-deployed, full-stack industrial bid management platform built with modern enterprise technologies



Backend

Spring Boot 3.2.5 ·
Java 17 · Spring
Security · JWT ·
MongoDB · WebSocket
· Docker



Frontend

React 19 · Vite ·
Zustand · Axios ·
Tailwind CSS ·
Recharts · React
Leaflet · xterm.js



Cloud & AI

MongoDB Atlas ·
Google Gemini AI ·
Google OAuth ·
OpenStreetMap ·
Render.com · Gmail
SMTP



Security

JWT HS256 · BCrypt-
12 · RBAC · CORS ·
OTP Email · Spring
Security Filter Chain



AI Features

Win Prediction ·
Sentiment Analysis ·
Executive Summary ·
Chat Assistant · Smart
Priority ·
Recommendations



Extra Features

3 Languages ·
Dark/Light Mode · Live
Map · SSH Terminal ·
Real-time Notifications
· Subscription Plans



Live Demo URL

<https://bidflow-frontend-jeeu.onrender.com>

GitHub: github.com/vishal-q/Bid_AuctionX_Tracking

BidFlow AuctionX Tracking — Project Documentation v1.0.0 · June 2026

Built with Spring Boot + React.js + MongoDB Atlas + Google Gemini AI